

# 基于Unity的游戏软件设计

AI NPC 与可验证动态事件系统

导师：朱正礼

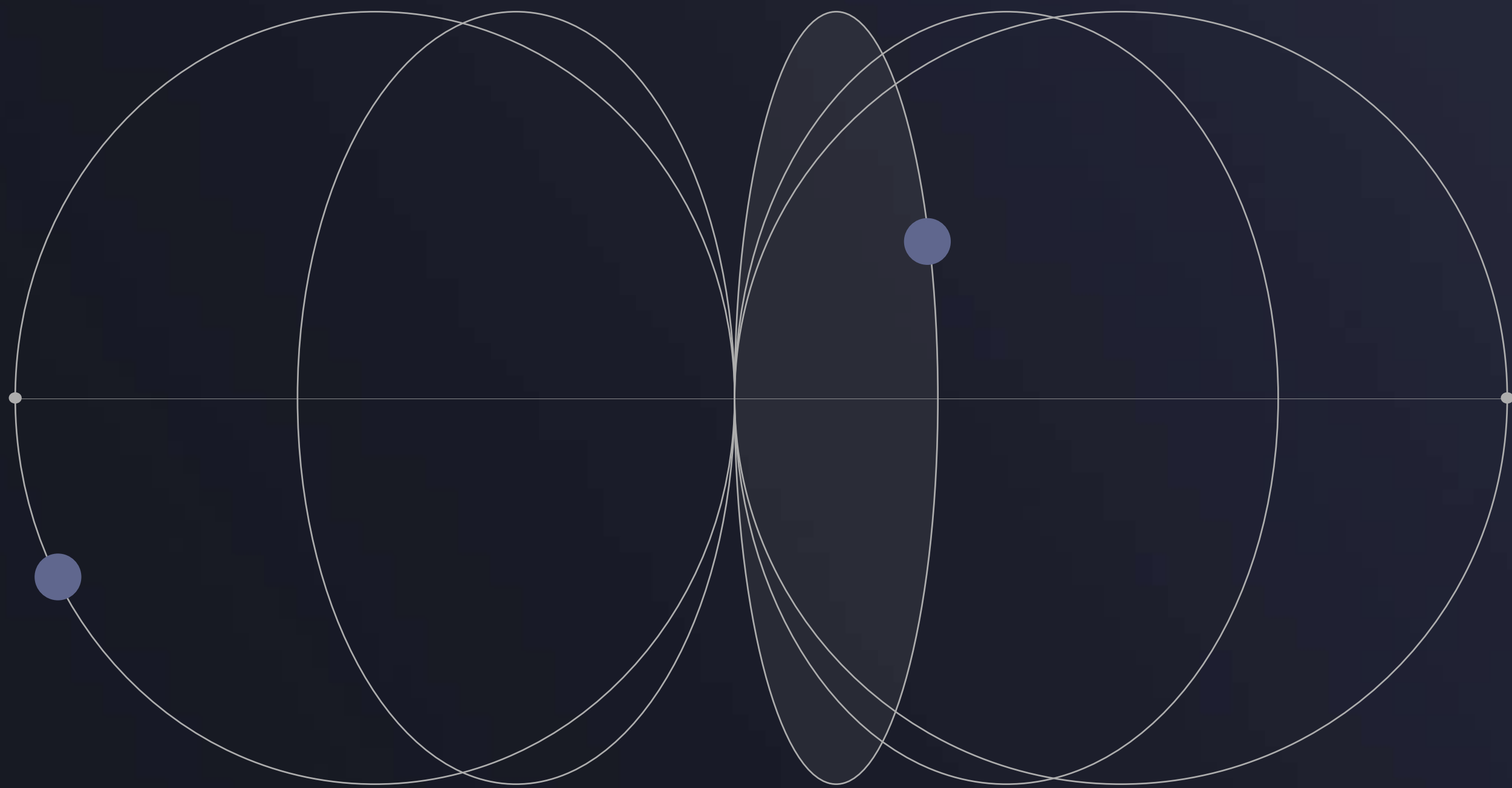
汇报人：王光耀

时间：2026/05/18



CONTENTS

目录



|           |    |           |    |
|-----------|----|-----------|----|
| 核心问题与项目定位 | 01 | 渔闻系统与玩法验证 | 04 |
| 技术矛盾与解决路线 | 02 | 扩展性与创新总结  | 05 |
| 系统架构与核心模块 | 03 |           |    |



01

SECTIONS

核心问题与项目定位

---



# 核心问题



## 传统NPC系统的局限

传统 NPC 稳定但固定，AI 对话自然但难以直接参与玩法。

接入大模型能够提升 NPC 对话的自然度，但它仍然只停留在聊天展示层。



# 项目定位

从聊天层到玩法层的跨越

本项目核心命题是：如何让 AI NPC 生成内容从“聊天文本”转化为可识别、可执行、可验证的游戏事件。



# AI动态事件闭环的三大流程节点

AI生成候选线索

本地校验与玩家采信

玩法验证与世界反馈

1

AI生成带有地点、目标、时间窗口和可信度的候选线索，为玩家提供动态信息来源。

2

本地系统对候选内容解析校验并注册，玩家主动采信后进入真实玩法验证。

3

Unity根据钓鱼结果验证事件真伪，公告板将结果反馈为世界信息，形成完整闭环。



02

SECTIONS

技术矛盾与解决路线

---



# 游戏确定性与AI不确定性的工程调和

## 自由发挥风险

AI 可能生成本地世界中不存在的地点、目标或事件。

通过本地世界数据限制生成范围。



AI 生成内容如果无法被验证，只能停留在剧情文本层。

系统需要接入真实玩法结果，对事件真伪进行验证。

## 内容验证缺失



## 自然语言解析难题

AI 输出如果停留在自然语言层，难以直接驱动游戏逻辑。

后端要求生成结构化事件字段，供 Unity 稳定解析。



AI 直接改状态会破坏规则确定性。

AI 仅生成候选事件由本地系统裁决。

## 状态修改风险





# 从玩家请求到结果反馈 的七步技术链路



- 1 玩家提出开放性请求
- 2 本地规则判断是否需要 AI
- 3 C++ AI 网关注入 NPC 与世界上下文
- 4 模型生成结构化候选事件
- 5 事件协议校验与本地注册
- 6 玩家通过真实玩法验证事件
- 7 结果反馈到 UI 与世界信息面板





03

SECTIONS

系统架构与核心模块

---



# 面向AI动态事件的四层工程架构设计





# 意图路由层：从自然语言输入到系统意图

## 核心架构



- step1: 玩家自然语言输入
- step2: 文本归一化
- step3: 意图解析服务
- step4: 输出结构化意图
- step5: 分发到对应系统

## 核心原则



- 本地强规则层——优先级最高，触发真是游戏状态，例如：采信渔闻、汇报验真失败
- NPC 规则回复层——适合处理稳定、可枚举的NPC功能，例如：湖区介绍、鱼价说明
- 后端大模型层——适合处理开放式自由对话，例如：“这里发生过什么”“你怎么看这片湖”
- 玩法服务层——真正修改系统状态的地方，例如：负责渔闻记录、验真状态

## 流程演示

玩家输入：  
这条渔闻不对

文本归一化：  
这条渔闻不对  
命中信息：  
包含“渔闻”“不对”

路由结果：  
intent = FailedRumorFeedbacks  
houldUseBackend = false  
reason = 命中“渔闻”上下文 + “不对”失败反馈词

## 意图解析服务数据结构

intent: 系统意图  
confidence: 置信度  
reason: 命中原因  
shouldUseBackend: 是否请求后端模型  
chatMode: 后端对话模式



# AI 网关层——从系统意图到候选事件

1

网关必要性

隔离模型接口细节，避免 API Key、协议差异和网络错误散落在 Unity 脚本层。

2

请求控制与  
业务编排

ChatController 负责请求解析与参数校验；  
ChatService 负责模式判断、任务组织和异常兜底。

3

Prompt 构造与  
模型调用

Builder 注入 NPC、湖区和鱼类上下文；  
Client 负责模型调用、协议适配和超时处理。

4

输出解析与  
稳定响应

Parser把不稳定的模型文本输出转化为 Unity 可消费的稳定JSON，作为候选动态事件进入前端玩法系统



# 结构化输出与本地约束层：从模型输出到可注册事件

## Prompt 侧约束：限定模型生成边界

PromptBuilder 在请求模型前注入本地上下文，包括 NPC 身份、玩家问题、已知湖区列表（known\_lakes）和已知鱼种列表（known\_fishes）。把模型生成空间限制在游戏世界已存在的数据范围内，减少凭空编造。

## Parser 侧校验：稳定模型输出结果

ModelOutputParser会提取 JSON，解析结构化字段，并对异常值进行归一化处理。例如：未知 emotion 回退为 neutral，未知 animation 回退为 idle。

## Unity 本地裁决：控制玩法状态

AI 网关只生成候选事件，不直接修改游戏状态。渔闻是否被采信、是否验真成功、是否发放奖励、是否影响鱼类概率，均由 Unity 本地玩法系统裁决。



04

SECTIONS

渔闻系统与玩法验证

---



# 从湖边传闻到可验证玩法事件的闭环

## 渔闻系统核心定义 01

渔闻不是AI写的钓鱼文案，而是转化为有地点、目标、时间窗口和可信度的候选事件。

## 完整闭环流程 02

玩家询问渔闻后AI生成结构化传闻，经校验注册并公告板发布，玩家采信后进入钓鱼验证。

## Gameplay Verifier验证逻辑 03

验证器不依赖AI判断，而是检查钓鱼结果是否满足湖区、鱼种、时间窗口匹配及次数上限和超时条件。

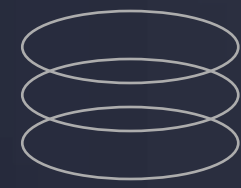


# 不确定但可验证的 玩法设计哲学



## 五大核心价值

渔闻是可采信的信息线索而非任务指令；玩家依据可信度自主决定投入；时间窗口赋予事件行动节奏；真实钓鱼结果赋予传闻可证伪性；验真或失准结果通过公告板形成世界反馈，构建动态信息生态。



## 设计哲学本质

可玩性来自不确定但可验证：模型提出线索，事实由Unity本地玩法系统产生。AI输出从确定性指令转变为信息驱动的探索动机和策略资源，赋予玩家自主判断和选择空间。



05

SECTIONS

扩展性与创新总结

---



# 从钓鱼渔闻到通用动态事件系统

## 可迁移玩法矩阵

1

架构可迁移至狩猎传闻（区域、动物、危险等级验证猎获）、采集线索（资源点、类型、刷新时间验证采集）、商人消息（商品、地区、价格波动验证经济变化）、任务提示（NPC、地点、目标条件验证任务完成）。

## 架构复用与替换点

2

意图路由、策略服务、候选事件生成器、协议校验器、动态事件注册表和结果分发器均可复用；不同玩法只需替换Gameplay Verifier验证逻辑，实现低成本扩展。



THANKS  
感谢观看